

PROGRAMMING II

CLASS CONSTRUCTORS AND ENCAPSULATION

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

CLASS CONSTRUCTORS



CLASS CONSTRUCTOR

To create a `Clazz` object in memory we use `Clazz myClazz = new Clazz();`

What happens behind the scene

- ▶ Some memory large enough to store a `Clazz` object is allocated in the heap
- ▶ A reference (pointing to the freshly allocated memory) is provided in `myClazz`
- ▶ All fields of the object pointed by `myClazz` are initialized
- ▶ The constructor of `Clazz` is called on the object pointed by `myClazz`

CLASS CONSTRUCTOR (CONT'D)

A class **constructor** is a special method

- ▶ Having the same name of the class
- ▶ Having no return value (**void** neither)
- ▶ That is executed only at class instance creation
- ▶ It is used to provide custom initialization to class fields

Indeed, a constructor can have zero, one or more parameters

PARAMETERIZED CONSTRUCTOR

A **parameterized constructor** is a constructor having parameters

```
public class Car {                                1
    String color;                                  2
                                                    3
    Car(String color) { // Parameterized constructor 4
        paint(color); // Methods can be called by constructors 5
    }                                              6
                                                    7
    void paint(String color) {                     8
        this.color = color;                       9
    }                                             10
}                                              11
```

We can create a Car object with `Car myCar = new Car("yellow");`

DEFAULT CONSTRUCTOR

If no constructor is given the compiler will add a **default constructor**

```
public class Car {  
    String color;  
  
    Car() { } // Default constructor  
  
    void paint(String color) {  
        this.color = color;  
    }  
}
```

1
2
3
4
5
6
7
8
9

This is why we are able to create a Car object with `Car myCar = new Car();`

FALSE CONSTRUCTOR

If a return value is provided we have a **false constructor**

```
public class Car {                                1
    String color;                                  2
                                                    3
    void Car(String color) { // The default constructor is added and 4
        this.color = color; // executed in place of this method      5
    }                                              6
                                                    7
    void paint(String color) {                      8
        this.color = color;                        9
    }                                              10
}                                                  11
```

void Car(String color) is seen as a normal method (not called ad instantiation)

A **copy constructor** initializes an object by using another object of the same class

- ▶ Java does not provide copy constructors, programmers should provide them

```
1 public class Car {
2     String color;
3
4     Car(Car carObj) { // Copy constructor
5         this.color = carObj.color;
6     }
7
8     void paint(String color) {
9         this.color = color;
10    }
11 }
```

CONSTRUCTOR OVERLOADING

A class can be initialized in multiple ways, by exploiting **overloading**

```
public class Car {                                1
    String color;                                  2

    Car(String color) {    // Parameterized constructor  3
        this.color = color;                                4
    }                                                    5
                                                    6
    Car(Car carObj) {    // Copy constructor              7
        this.color = carObj.color;                        8
    }                                                    9
                                                    10
    void paint(String color) { this.color = color; }     11
}                                                        12
                                                    13
```

We can do `Car myCar1 = new Car("red");` and `Car myCar2 = new Car(myCar1);`

```
1 public class Car {
2     String color;
3
4     Car(String color) {    // Parameterized constructor
5         this.color = color;
6     }
7
8     Car(Car carObj) {    // Copy constructor
9         this(carObj.color);    // calling the parameter constructor
10    }
11
12    void paint(String color) { this.color = color; }
13 }
```

OVERLOADING

Math major:



**abuse
of notation**

Programmer:



**operator
overloading**

OBJECT DESTRUCTOR

In Java, programmers have not to care about memory de-allocation

- ▶ But the language allows to execute code at garbage collection
- ▶ By implementing a custom `finalize()` method *// deprecated from Java 9*

```
public class Car {                                1
    String color;                                  2

    void paint(String color) { this.color = color; } 3
                                                    4

    protected void finalize() { // Do not use finalizers! 5
        System.out.println("Farewell_" + this);      6
    }                                                  7
}                                                      8
                                                    9
```

ENCAPSULATION



ENCAPSULATION

Encapsulation: bundling data (fields) and operations (methods) acting on that data into a single unit (class)

```
public class Person {                                1
    String name;                                     2
    int age = -1;                                    3

    Person(String name) { this.name = name; }        4
                                                    5
    void changeAge(int age) {                        6
        if (age > 0 && age < 122) this.age = age;    7
        else System.out.println("Invalid age.");    8
    }                                                9
}                                                  10
                                                    11
```

Ok, but we can still do `Person sam = new Person("Sam"); sam.age = -2;`

INFORMATION HIDING

Information hiding: restrict direct access to an object's data and implementation details from the outside world

We can have it by coupling encapsulation with **access modifiers**

- ▶ Encapsulation provides modularity
- ▶ Access modifiers restrict access to data

Some data can be accessed only by class instances (**private**) while other data can be accessed also by the outside world (**public**)

INFORMATION HIDING (CONT'D)

Information hiding: restrict direct access to an object's data and implementation details from the outside world

```
public class Person {                                1
    String name; // The default visibility is public  2
    private int age = -1;                             3
                                                    4

    Person(String name) { this.name = name; }         5
                                                    6

    void changeAge(int age) {                          7
        if (age > 0 && age <= 122) this.age = age;    8
        else System.out.println("Invalid age.");     9
    }                                                  10
}                                                       11
```

Now `Person sam = new Person("Sam"); sam.age = -2;` yields a compilation error

INFORMATION HIDING (CONT'D)

Information hiding: restrict direct access to an object's data and implementation details from the outside world

It allows to

- ▶ Write modular and reusable code
- ▶ Enforce integrity of internal data
- ▶ Enforce confidentiality of implementation details
- ▶ Facilitate code evolution

Interface programming: only the exposed interface is needed to use a component, internal data and implementation may freely change

INFORMATION HIDING (CONT'D)

**Most software is
written like an onion**



**The more layers you peel
back, the more you want to cry**

SETTERS AND GETTERS

Private fields can (or cannot) be visible to the outside world

```
public class Person { 1
    private String name; 2
    private int age = -1; 3
    4
    Person(String name) { this.name = name; } 5
    6
    String getName() { return name; } 7
    8
    int getAge() { return age; }; // the default visibility is public 9
} 10
```

We cannot do `samAge = sam.age;` but we can do `samAge = sam.getAge();`

SETTERS AND GETTERS (CONT'D)

For each private field, provide a setter and a getter method *// not mandatory*

```
public class Person {                                1
    private int age = -1;                             2

    // Setter                                           3
    void setAge(int age) { // Controlled age update    4
        if (age > 0 && age <= 122) this.age = age;      5
        else System.out.println("Invalid age.");       6
    }                                                    7
                                                        8
    // Getter                                           9
    int getAge() { return age; } // Make age available to the outside 10
}                                                         11
                                                         12
```

You can have multiple setters (overloading)

ADVANTAGES OF SETTERS AND GETTERS

Setters allow performing checks before modifying fields

- ▶ Validation rules, authorization, etc.

Getters allow changing internal representation without affecting the codebase

- ▶ Type conversion to keep the interface unchanged

```
public class Person {  
    private short age = -1; // more memory-efficient  
  
    // Backwards-compatible getter  
    int getAge() { return age; } // Implicit type conversion  
}
```

1
2
3
4
5
6

METHOD ACCESS

We can apply access modifiers to methods as well

- By default, all methods are **public**

```
public class Person {  
    private String name;  
    private int age = -1;  
  
    public Person(String name) { setName(name); }  
  
    // Setters  
    private void setName(name) { this.name = name; } // Differently from age, name can  
    public void setAge(int age) { // only be set at instantiation  
        if (age > 0 && age <= 122) this.age = age;  
        else System.out.println("Invalid age.");  
    }  
  
    // Getters  
    public String getName() { return name; }  
    public int getAge() { return age; }  
}
```

ACCESS MODIFIERS

A **public** field or method

- ▶ Can be accessed by methods of the same class
- ▶ Can be accessed by methods of other classes

A **private** field or method

- ▶ Can be accessed by methods of the same class
- ▶ Cannot be accessed by methods of other classes

ACCESS MODIFIERS



TESTING

Implemented functionalities **should** be tested

- ▶ Java provides the `assert` keyword to check conditions at runtime
- ▶ Introduced to verify that some working conditions are met
- ▶ Assertions must be enable in the JVM with `java -ea`

```
if (person == null) { ... } // handle error
else { ... } // continue nominal execution

// shorter replacement
assert person != null;
// nominal execution
```

1
2
3
4
5
6

TESTING (CONT'D)

Implemented functionalities **should** be tested

- We will see more advanced testing techniques later in the course

```
public class TestPerson { // test class      1
    private static Person person = new Person("Sam");      2
                                                                3
    private static void setAgeTest() { // test method      4
        person.setAge(0);      5
        assert person.getAge() != 0 : "Age cannot be 0"; // error message      6
    }      7
    // run with -ea JVM option      8
    public static void main(String[] args) { setAgeTest(); }      9
}      10
```

Q + A

